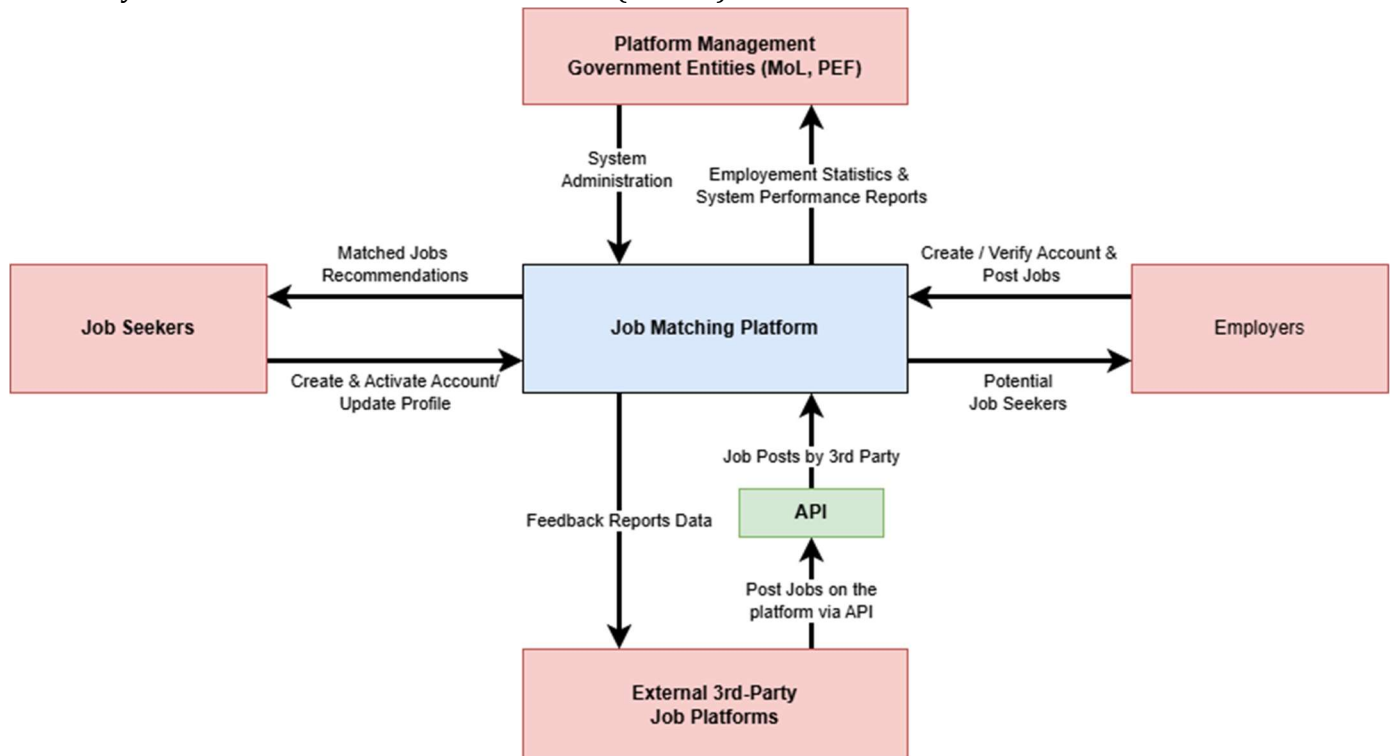


6.4. Appendix A: Analysis Models

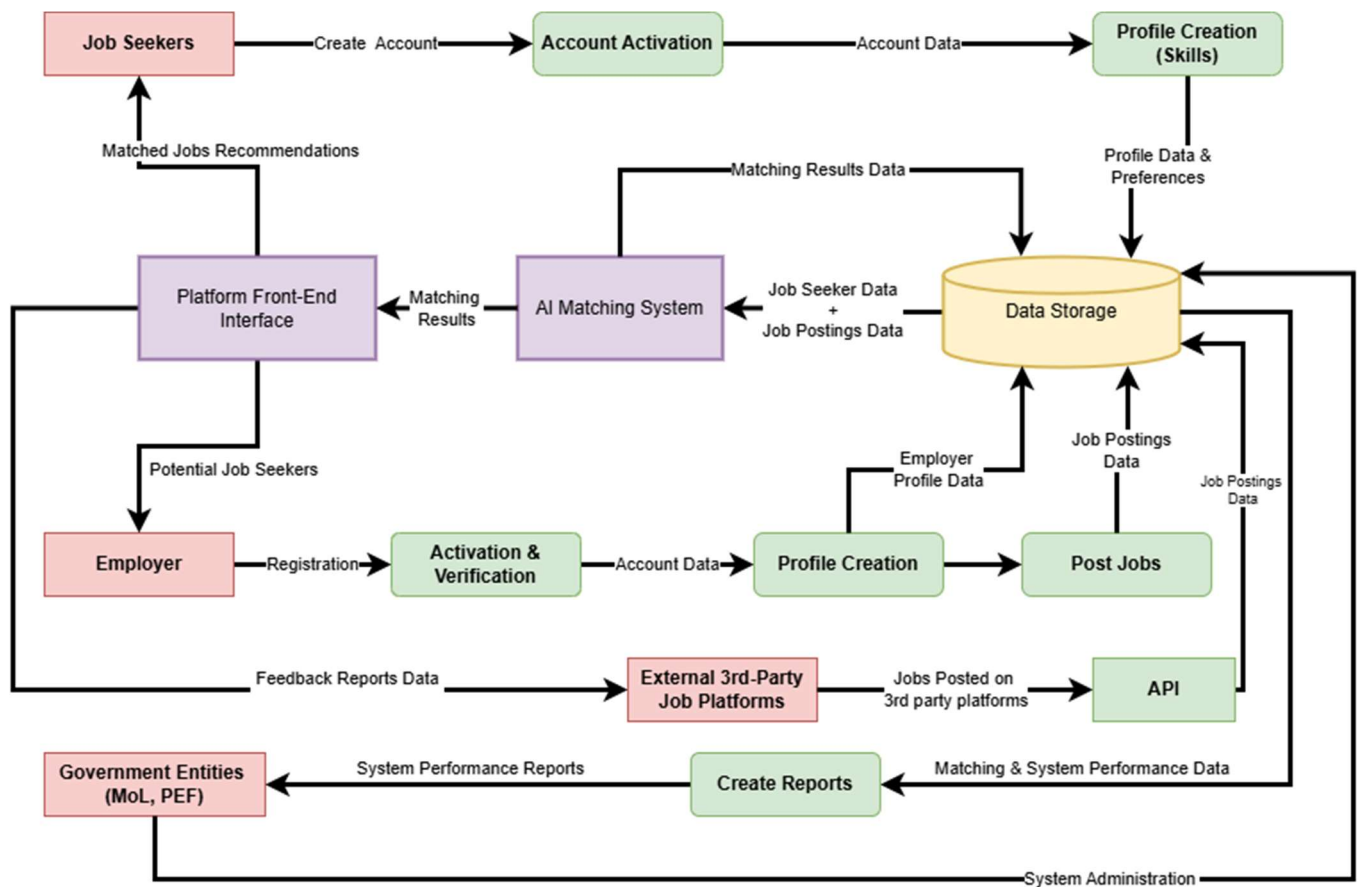
This appendix will include the following analysis models:

1) Context Diagram

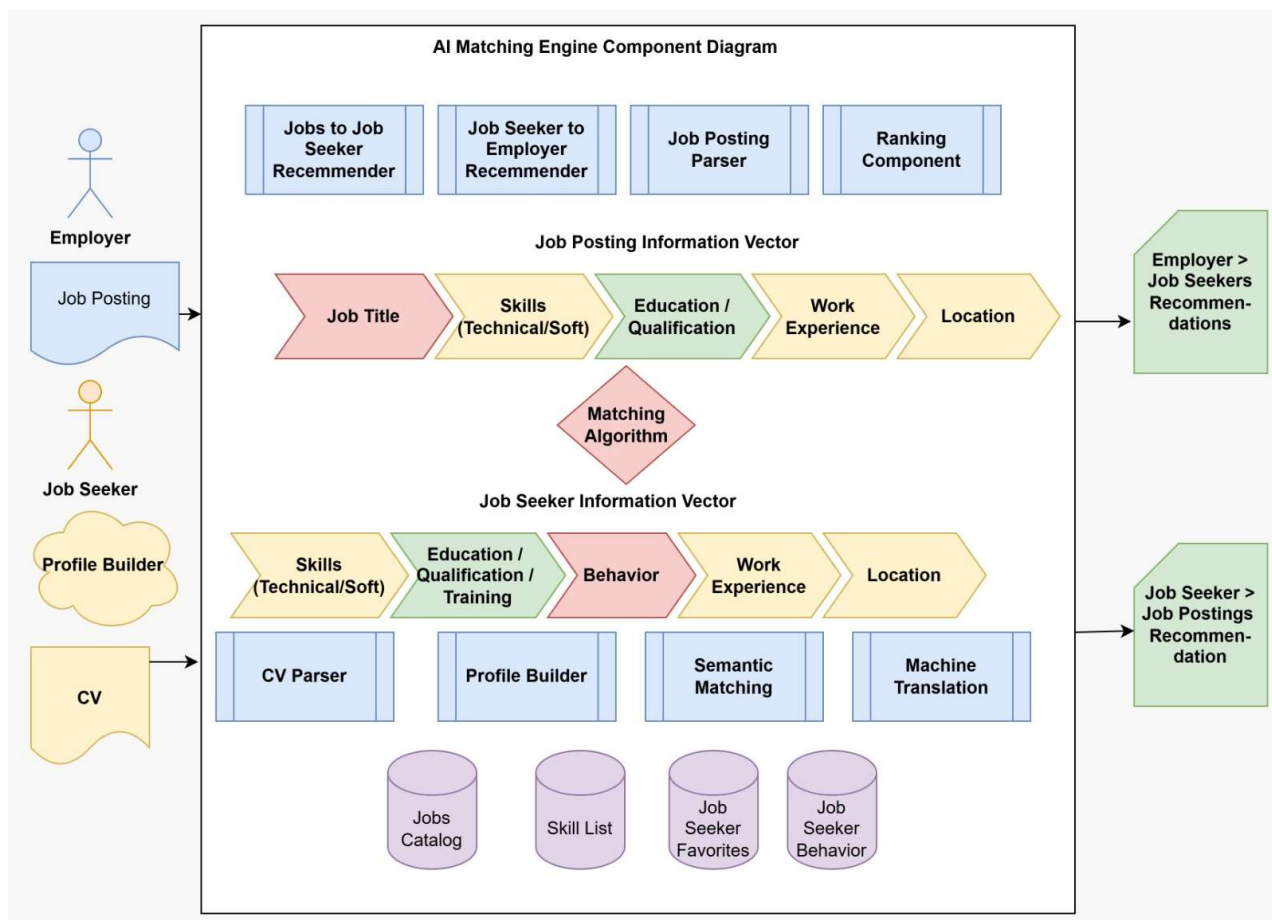
- System boundaries and external entities (Level 0)



- Data flows between the system and external entities (Level 1)

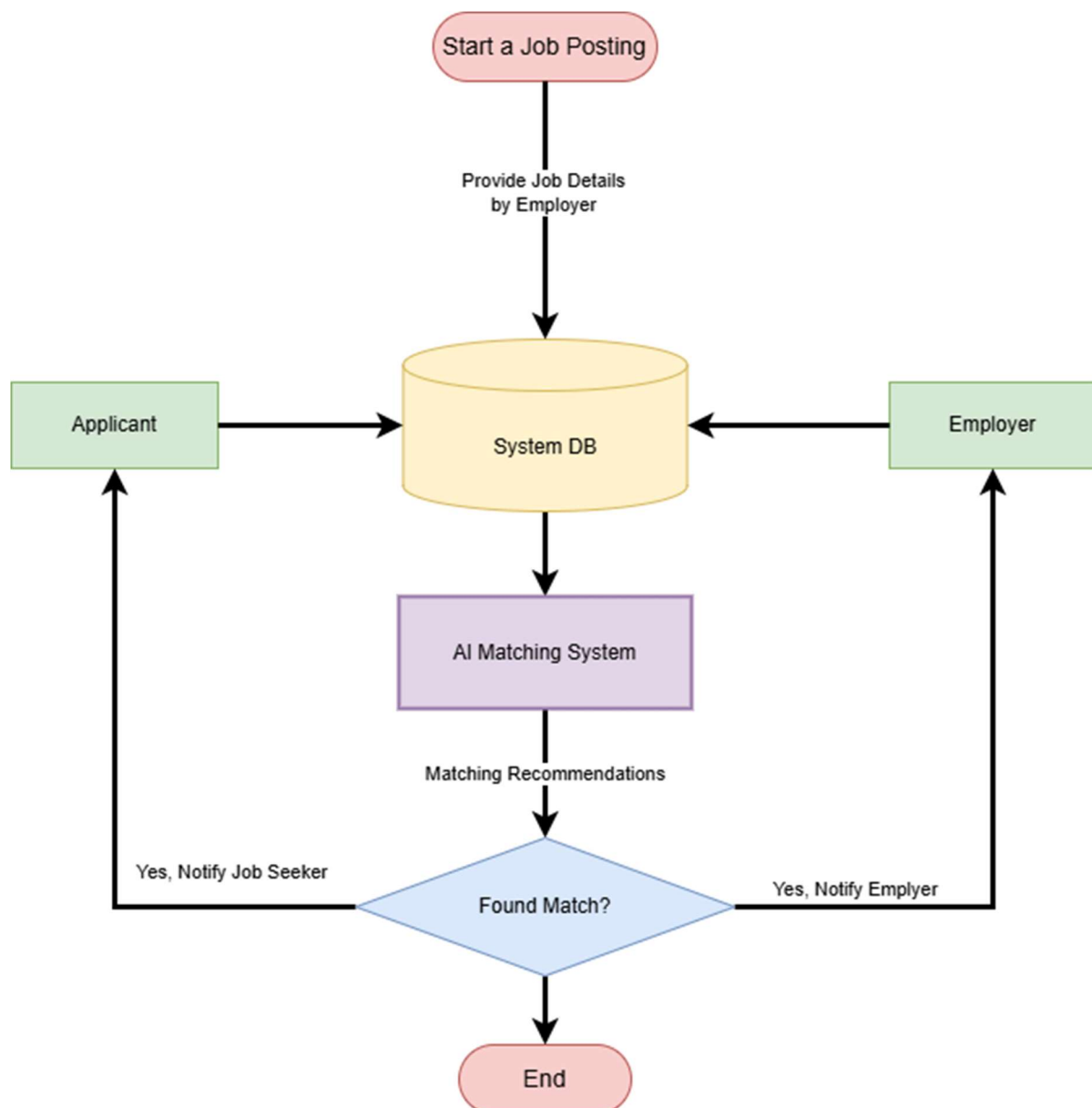


2) AI Engine Component Diagram

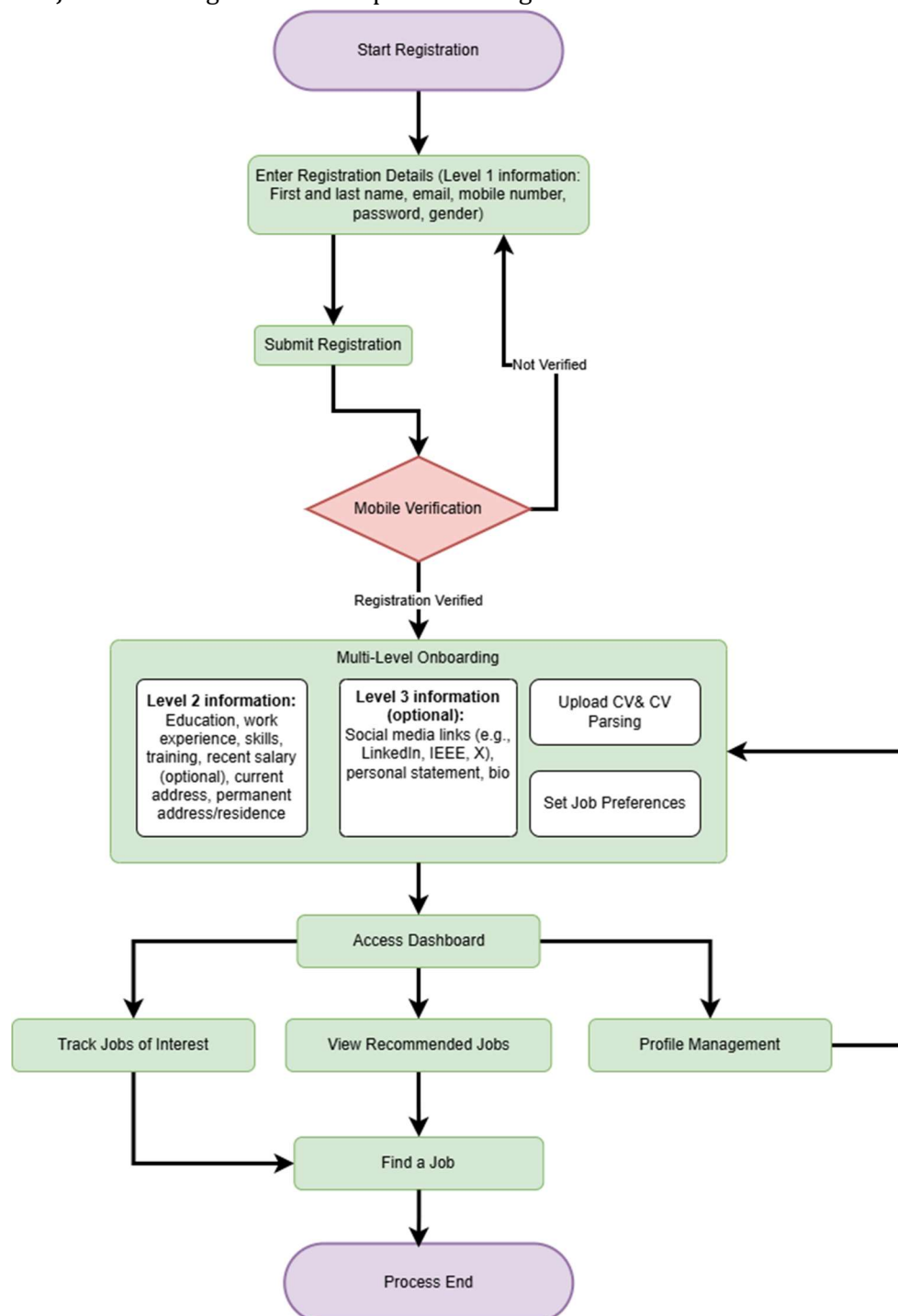


3) Process Flow Diagrams

- Job posting and application process



- User registration and profile management
 - Job seeker registration and profile management



- Employer registration and profile management

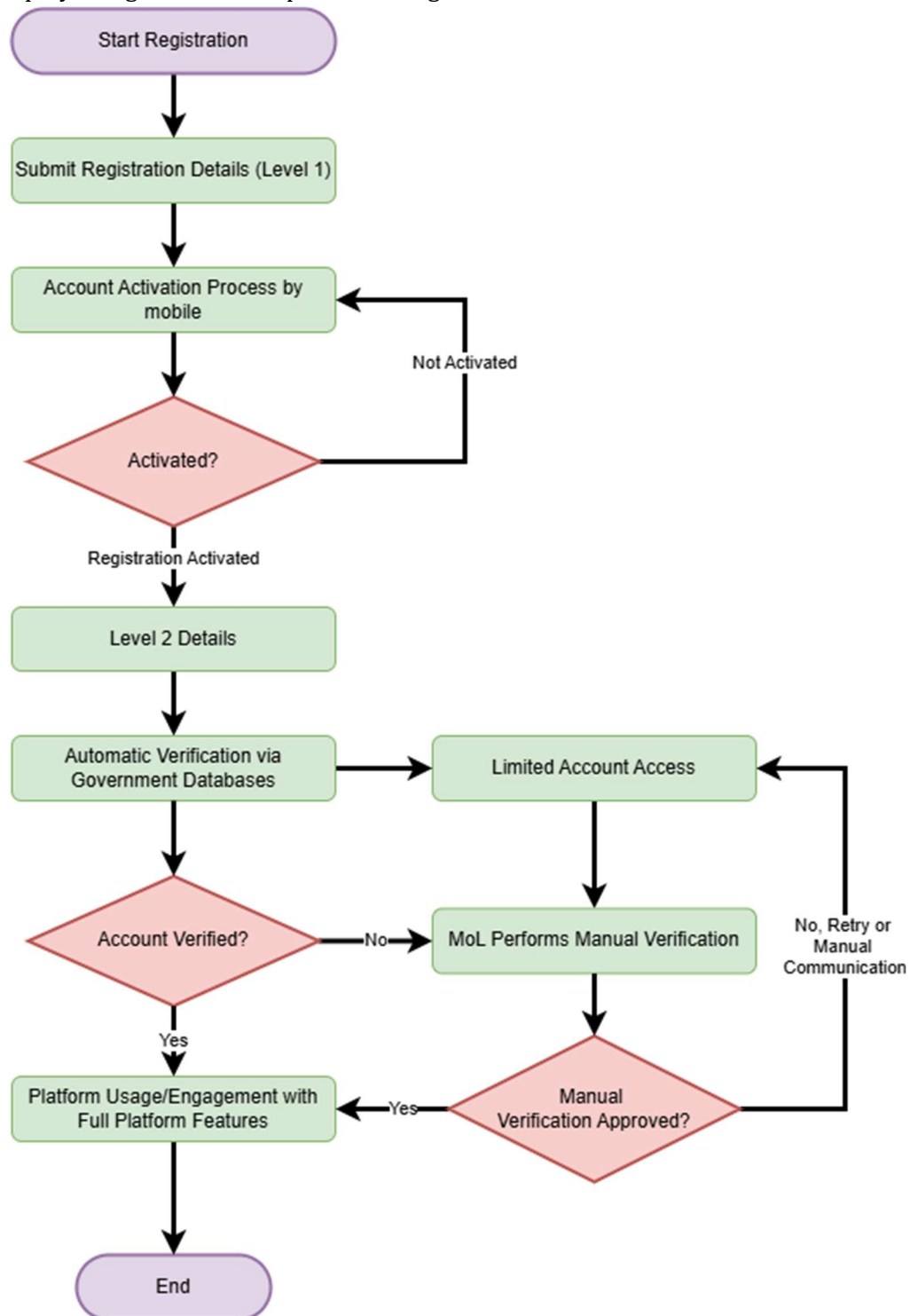


Figure 3-1 Sample Schemas HR and OE

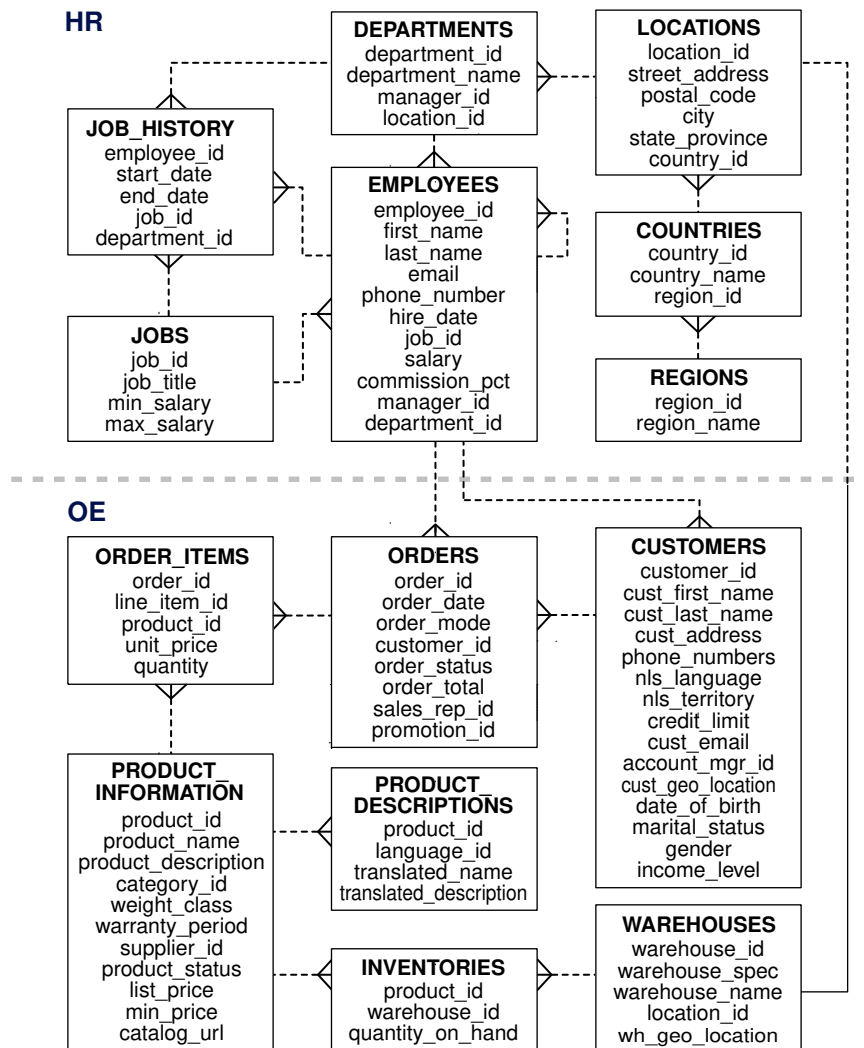


Figure 3-2 Sample Schemas OE and PM

OE



PM

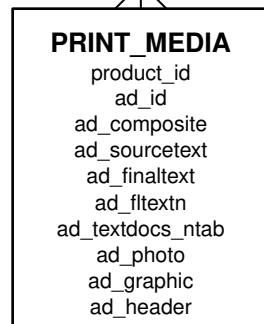


Figure 3-3 Sample Schema SH

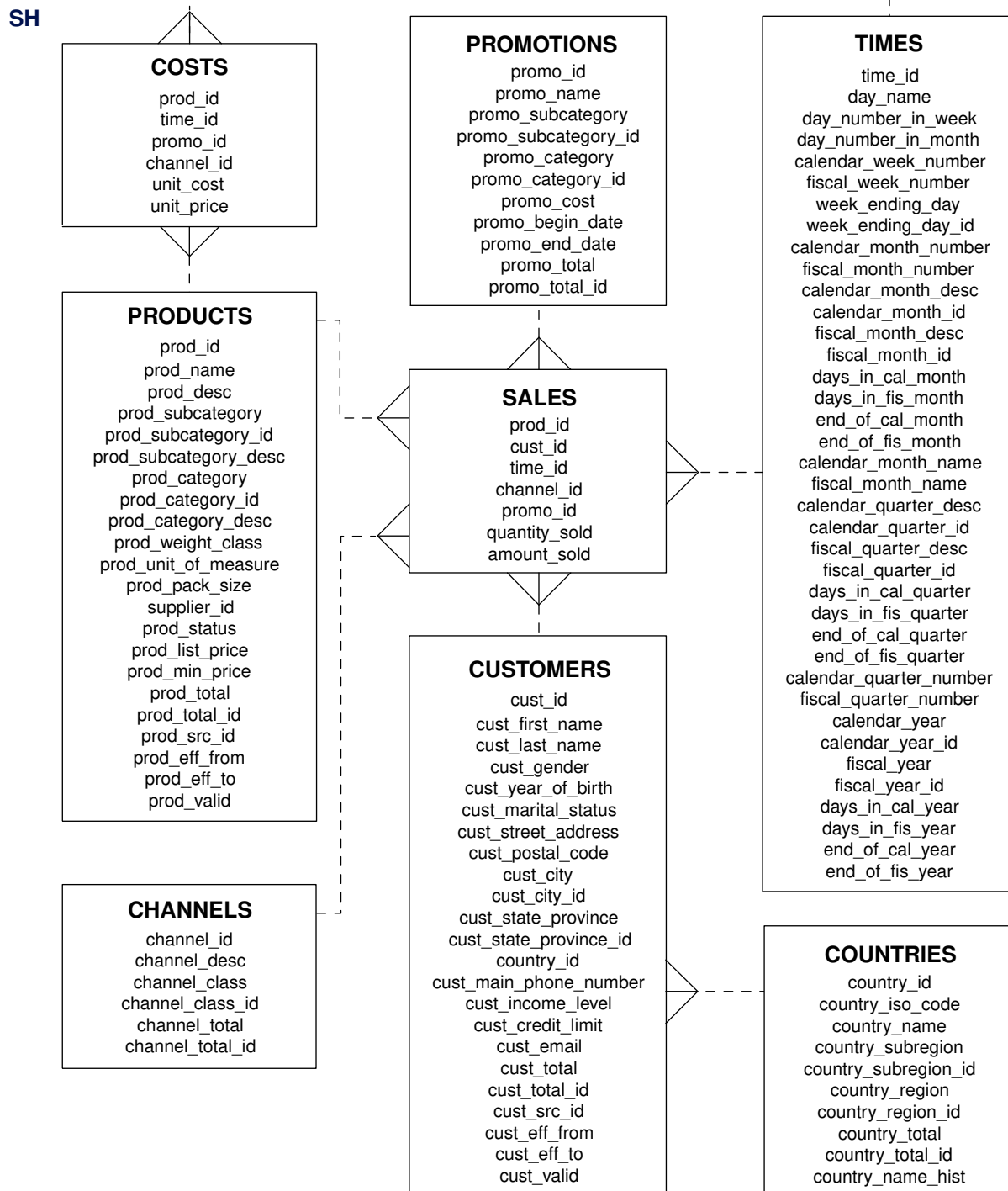
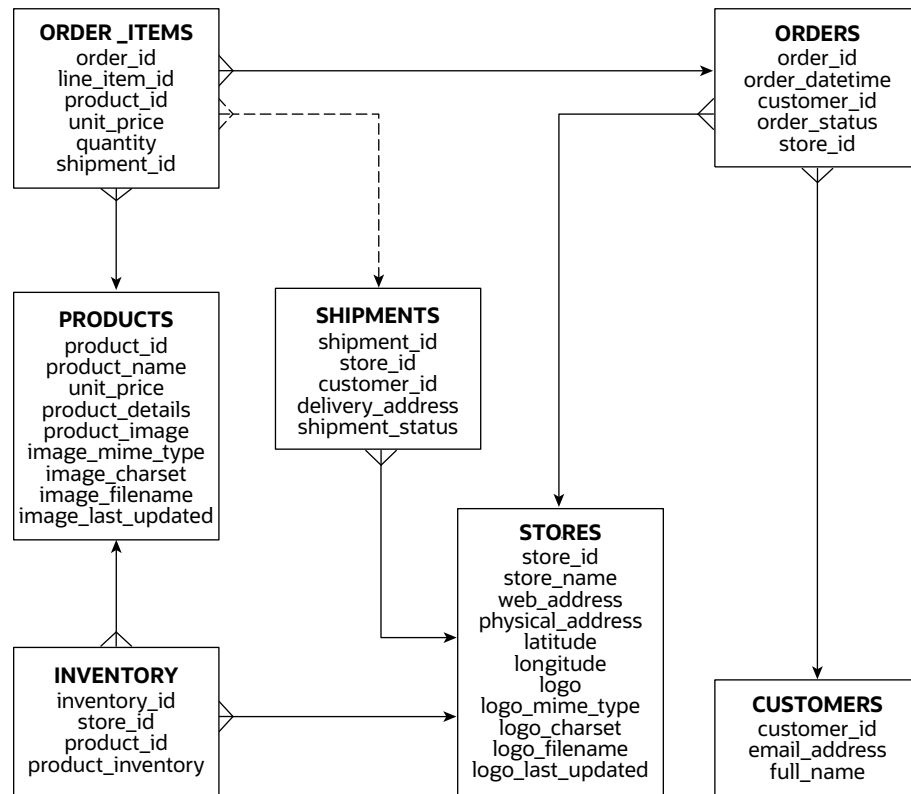


Figure 3-4 Sample Schema CO

CO



An API Sandbox Test Form

GET

/api/campaigns

Return info for campaigns. Filters can be applied.

Documentation

Sandbox

Input

Filters

page

=

Optional. Page of result

-

created_since

=

Optional. Return campa

-

created_before

=

Optional. Return campa

-

Try!

Headers

x-api-key

=

INuQeqoelZ1kmLWRPcTh

-

New header

Content

Content set here

will override the

parameters that do

<

>

//

Content-Type

=

Value

Set header

Replaces header

if set

4. Click **Try!** to execute the API request.

5. The display shows results such as the following:

Try!

Request URL

GET /api/campaigns

Response Headers [Expand]

200 OK

Response Body [Raw]

```
{
  "id": 17,
  "title": "Aspiring Director's Fund",
  "details": "<p><span style='\"color:rgb(0, 0, 0); font-size:14px\">The Motion Picture Association (MPA) and the Asia Pacific Screen Awards (APSA) gives an opportunity to aspiring filmmakers with the MPA APSA Academy Film Fund.&nbsp;</span><em>MPA APSA Academy Film Funds&nbsp;</em><span style='\"color:rgb(0, 0, 0); font-size:14px\">is a unique initiative which has aided aspiring film makers in a lot of development in new film projects.</span></p>",
  "goal": 501,
  "thank_you_message": "Dear #name#,\r\n\r\nThank you for your $#amount# donation to the #title# campaign! We truly appreciate your support.\r\n\r\nEvery bit helps, and you've taken us one step closer to achieving our goal.\r\n\r\nCheck back often to see which of your friends have donated, and the progress we've made. \r\n\r\nThanks for helping us to make this our most successful year yet!\r\n\r\nSincerely, 360Alumni Support",
  "type": "short",
  "start_date": "2013-11-15 00:00:00",
  "end_date": "2017-01-31 00:00:00",
  "createdAt": "2013-11-15 17:33:07",
  "updatedAt": "2016-06-16 11:42:09",
  "isInactive": null,
  "author": {
    "user_id": 1092,
    "first_name": "Georgette",
    "last_name": "Lucas"
  },
  "raised": "465",
  "image": "/cdn/images/f19cb30cf19d512a9de8f487a5bc1d13.jpg",
  "ambassadors": []
}
```

Curl Command Line

```
curl -X "GET" -H "Accept:\ application/json" -H "Content-type:\ application/json" -H "x-api-key:\ <<your-API-key>>" /api/campaigns
```

Note the following about the results:

- **Request URL:** Shows what executed.
- **Response Headers:** Contains messages about the success or failure of the request.
- **Response Body:** Contains a JSON array of the results, in this case an array of [Campaign Objects](#). Other API requests use the [Donation Object](#) or the [User Object](#).
- **Curl Command Line:** Contains a form of the request that you can copy and paste to execute at the command line or in scripts.

```
curl -X "GET" -H "Accept:\ application/json" -H "Content-type:\ application/json" -H "x-api-key:\ <<your-API-key>>" /api/campaigns
```

6. Each campaign object in the returned array is identified with an **id** value. Note the first object in the array above contains the id value 17.

7. To get a single campaign object, click **GET /api/campaigns/{id}**.

This API request requires an integer **id** parameter that represents the campaign identifier.

The Reference Page for GET /api/campaigns/{id}

GET /api/campaigns/{id} Return info for a single campaign

Documentation Sandbox

Requirements

Name	Requirement	Type	Description
id	\d+	integer	campaign id

8. Click **Sandbox** and enter the following to execute the API request:

- Enter your API key in an HTTP request header.
- Enter an **id** parameter value representing the campaign you want to get.

C. Click **Try!** to execute the API request.

The Sandbox Form for a GET /api/campaigns/{id} Request

GET

/api/campaigns/{id}

Return info for a single campaign

Documentation

Sandbox

Input

Requirements

id = 17

Headers

x-api-key = <<your-API-key>>

New header

Content

Content set here will override the parameters that do

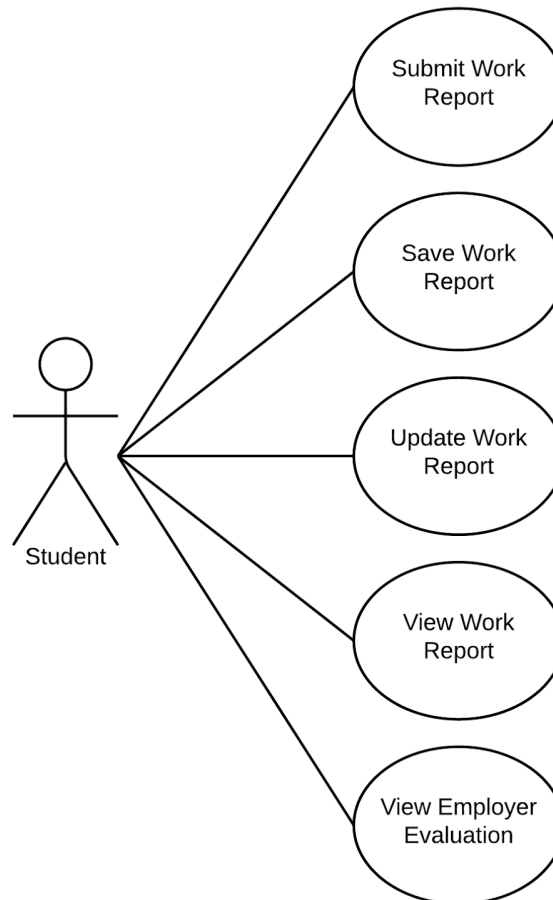
Content-Type = Value

Set header Replaces header if set

Try!

4.4 Student

4.4.1 Use Case Context

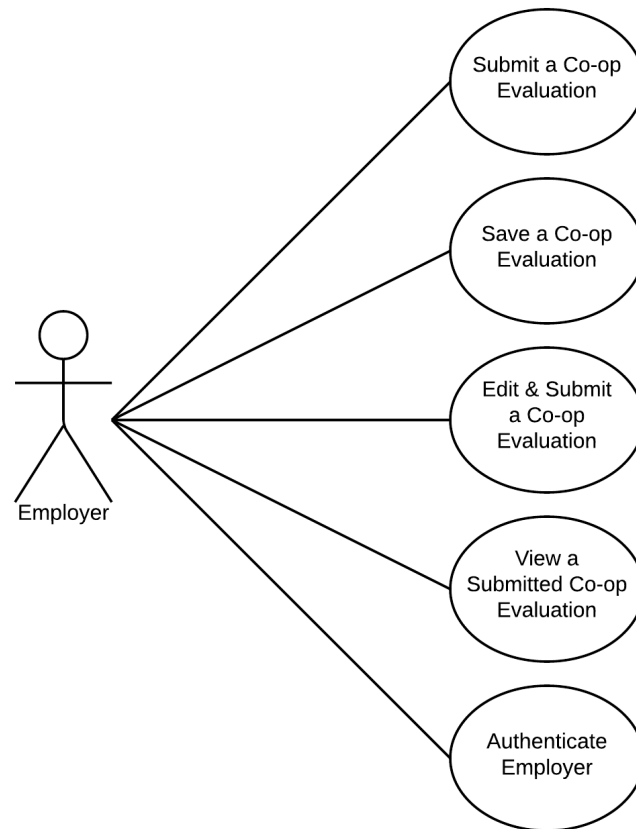


4.4.2 Submit Work Report

Actors	Student
Description	This use case describes how the Student uses the Co-op Evaluation System to submit a new Work Report.
Trigger	Student has a co-op for which a Work Report needs to be submitted.
Pre-conditions	1. The Student has an active Internet connection. 2. The Student has previously registered a co-op.
Post-conditions	A Work Report has been submitted for the respective Student's co-op.
Normal Flow	1. The Student locates the panel for the co-op in question.

4.3 Employer

4.3.1 Use Case Context



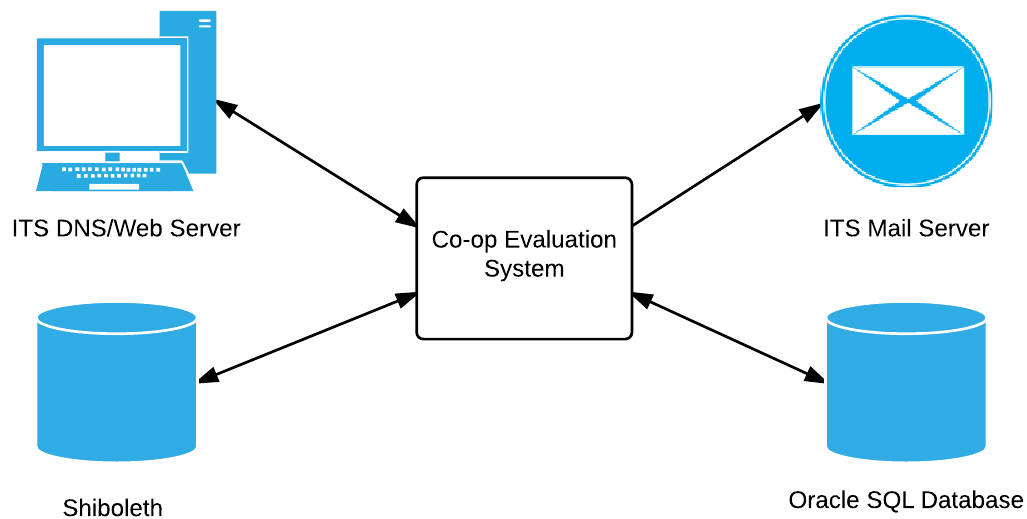
4.3.2 Submit a Co-op Evaluation

Actors	Employer
Description	This use case describes how the Employer uses the Co-op Evaluation System to submit a new Co-op Evaluation.
Trigger	Employer has a Co-op Student for which an Evaluation needs to be submitted and has received an email telling them the form is available.
Pre-conditions	1. The Employer has an active Internet connection. 2. The Employer has a registered Co-op Student.
Post-conditions	The Co-op Evaluation has been submitted for the Co-op Student.
Normal Flow	1. The Employer locates the panel for the student in question under “Current Co-op Students”.

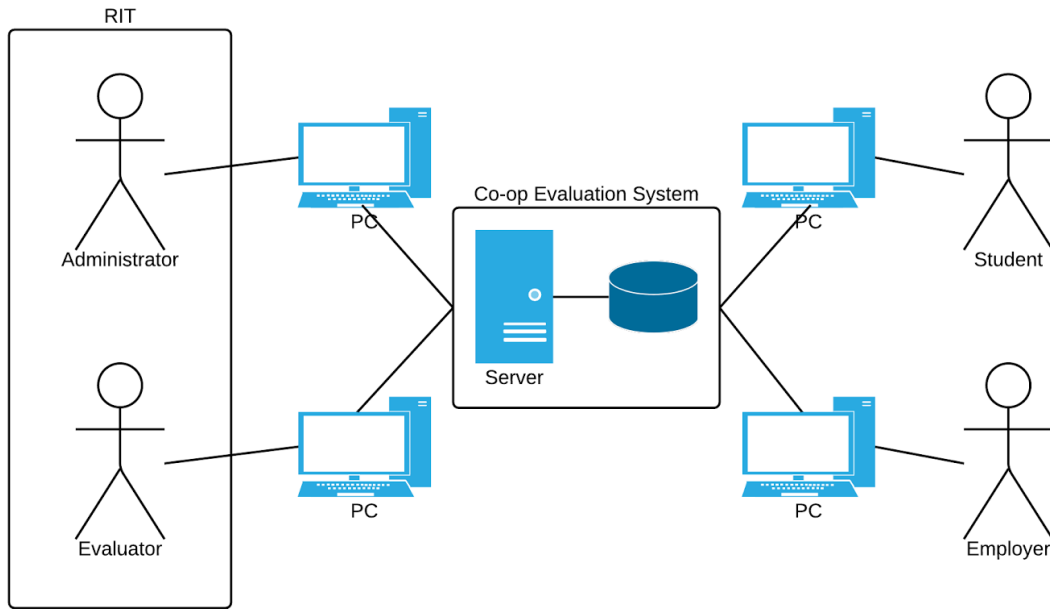
2 Overall Description

2.1 Product Perspective

The purpose of this project is to re-engineer the Co-op Evaluation System in order to leverage newer web technologies while also improving performance and user interaction. The current system uses outdated, under-documented technology, which makes it difficult to maintain. Furthermore, the random errors that occur do not give users confidence that their information was submitted properly. Significant improvements to the user interface will need to be made, but the existing database structures can be used as a reference for modifications.



The above diagram outlines the major components of the overall system, subsystem interconnections, and external interfaces, which are elaborated upon in Section 5.



The diagram above show a high-level view of the user interaction with the system as well as the interaction between technologies involved.

2.2 User Classes and Characteristics

User	Description of Use	Frequency of Use
Student	Uses application to fill out a Work Report	1 time per co-op block
Employer	Uses application to fill out an Employer Evaluation	1 time per student per co-op block
Evaluator	Uses application to review the student's and employer's survey responses to determine the student's grade (S or F)	1 time per student being evaluated
Administrator	Uses application to gather statistical data from survey responses	As needed to generate reports, create new forms, and perform other administrative tasks

2.3 Evaluation Status States

Evaluations, both Employer and Student, will start out as Pending. In the pending state, the evaluation appears in the system, but cannot be filled out yet.

Three weeks from the end of the term, all evaluations will be changed from Pending to Open. An evaluation in the Open state can be filled out by an Employer or Student that it is assigned to. An open evaluation can be changed to Saved, Submitted, Manually Completed, or Archived state.

If the user saved the evaluation, it is changed to the Saved state. In this state, the user can open the evaluation and continue filling out answers. They can continue saving the evaluation until they are finished. The Saved state will be referred to as “In Progress” in the user interface. The evaluation can be changed to Submitted, Manually Completed, or Archived state from the Saved state.

The Submitted state is reached by submitting the form from either the Open state or the Saved state. In this state, the evaluation cannot be changed by any class of user. At this point, the only change that can be made to the evaluation is by an evaluation approval change. If the evaluation is approved, nothing else will happen. If the evaluation is rejected, the evaluation will go back to the Saved state.

There are two other states that an evaluation can end up in: Manually Completed and Archived. If the evaluation is completed in some way other than the normal process, an evaluation can be changed to the Manually Completed state. If the evaluation will never be completed, and the user does not want to receive messages telling them to fill out the evaluation, the evaluation can be marked as Archived. The Archived state was known as the Past Pending state in the previous version of the CES.

